Семинар катедре за рачунарство

Квантно рачунарство и НП-тешки проблеми:

Од MaxCut проблема до аутоматског резоновања

Александар Ивановић*

Senior Software Engineer, Levi9

Катедра за рачунарство, Математички факултет

Универзитет у Београду

12. април 2026.

Информације

Догађај: Семинар катедре за рачунарство и информатику **Датум:** 12. април 2026. **Место:** Математички факултет, Универзитет у Београду **Тема:** Квантно рачунарство и примене

Сажетак

Овај рад разматра везу између аутоматског резоновања, QUBO моделирања и квантне оптимизације на примеру проблема MaxCut. Најпре је приказано како се MaxCut формулише у оквиру SMT решавача Z3, који у овом контексту служи и као егзактан решавач за мање инстанце и као основа за формалну проверу исправности модела. Затим је анализирана QUBO формулација као заједнички језик који повезује ограничења, класичне хеуристике и квантне оптимизационе алгоритме као што је QAOA.

*Аутор за контакт. Контакт: `aleksandar.ivke96@gmail.com`, `aleksandar.ivanovic@levi9.com`

Посебна пажња посвећена је томе како Z3 може да проверава семантику казнених чланова, избор коефицијената казне и исправност кандидатских решења добијених оптимизацијом. Тиме ћемо показати нове потенцијалне правце истраживања на пресеку формалних метода и квантних алгоритама.

Кључне речи: MaxCut, Z3, SMT, QUBO, QAOA, аутоматско резоновање, квантна оптимизација.

Садржај

1 Увод **4**

2 Квантно рачунарство и различите парадигме **4**
2.1 Заједничке основе: кубити и суперпозиција 4
2.2 Парадигма I: Квантна кола (Gate-Based QC) 5
2.3 Парадигма II: Квантно жарење (Quantum Annealing) 5
2.4 Поређење парадигми 7

3 Проблем MaxCut и НП-потпуност **7**
3.1 Класе сложености: P, NP, НП-тешко и НП-потпуно 8
3.2 Формална дефиниција 9
3.3 Илустративни пример 9
3.4 НП-потпуност проблема MaxCut 9
3.5 Апроксимациона сложеност 10

4 Класични приступ: Z3 решавач и SMT **10**
4.1 SAT и SMT решавачи 10
4.2 Z3: Архитектура и могућности 11
4.3 Кодирање MaxCut у Z3 11
4.4 Ограничења SMT/SAT приступа 12

5 QUBO: Quadratic Unconstrained Binary Optimization **13**
5.1 Дефиниција и основна структура 13
5.2 Примери QUBO формулација ограничења 14
5.3 Пример примене QUBO казни: тачно-2-од-3 ограничење 15
5.4 Аналогија са Лагранжевим множитељима 16
5.5 Формулација MaxCut-а као QUBO 16
5.6 Решавање QUBO: Z3 и симулирано жарење 18
5.6.1 Решавање QUBO помоћу Z3 18

5.6.2 Решавање QUBO симулираним жарењем (D-Wave Ocean SDK) 21

6 QAOA и MaxCut **22**

6.1 Од MaxCut-а до проблемског Хамилтонијана 22

6.2 Зашто је QAOA важан 23

7 Квантно рачунарство и аутоматско резоновање **23**

7.1 Z3 као верификатор QUBO ограничења 23

7.2 Предлог верификационог тока рада 27

8 Закључак **28**

1 Увод

Рачунарство је суочено са класом проблема које класични рачунари не могу да решавају ефикасно за велике улазе, такозваним НП-тешким проблемима. Ови проблеми јављају се у логистици, криптографији, машинском учењу и верификацији софтвера, па је њихово ефикасно решавање од велике практичне важности.

Квантно рачунарство нуди нову рачунску парадигму засновану на принципима квантне механике, суперпозицији и спрегнутости, која потенцијално омогућује истовремено претраживање експоненцијално великог простора стања. Квантна надмоћ над класичним рачунарима за НП-тешке проблеме теоријски није доказана, али хибридни квантно-класични алгоритми показују значајан практичан потенцијал.

У овом раду ћемо користи проблем *MaxCut* као водећи пример кроз који ћемо покушати да приближимо могуће предности једне гране квантног рачунарства. Представићемо модел **квантног жарења (Quantum Annealing)** и одговарајући математички формализам **квадратне неограничене бинарне оптимизације (QUBO quadratic unconstrained binary optimisation)** и на њему моделовати проблем *MaxCut*. Такође ћемо исти проблем моделовати коришћењем *Z3* решавача и указати на везу са *QUBO*, као и начине на који се може користити Z3 за верификацију решења добијених квантном оптимизацијом. Примери ће бити базични, али ће послужити као илустрација како се како се формална верификација може користити као алат за проверу квантних алгоритама.

2 Квантно рачунарство и различите парадигме

У овом раду ћемо споменути две парадигме које су релевантне за оптимизацију и каснију дискусију о QUBO формулацијама: **квантно рачунарство засновано на колима** и **квантно жарење**. Фокусираћемо се на кватно жарење као посебну парадигму за оптимизацију са добрим потенцијалом за решњавање комбинаторно оптимизационих проблема.

2.1 Заједничке основе: кубити и суперпозиција

Обе парадигме полазе од кубита, квантног аналога класичног бита. Стање регистра од n кубита описује се вектором у простору [9]

$$\mathcal{H} = (\mathbb{C}^2)^{\otimes n},$$

па опште стање има облик

$$|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle, \qquad \sum_x |\alpha_x|^2 = 1.$$

За разлику од класичног регистра, који је у сваком тренутку у једној конкретној конфигурацији, квантни регистар пре мерења може бити у суперпозицији више конфигурација истовремено. Мерење затим враћа један базни вектор $|x\rangle$ са вероватноћом $|\alpha_x|^2$.

Ова могућност није сама по себи *паралелно рачунање*; предност настаје тек када се амплитудама управља интерференцијом тако да се вероватноћа жељених стања појача, а нежељених потисне.

2.2 Парадигма I: Квантна кола (Gate-Based QC)

У моделу квантних кола, рачунање се изводи секвенцом унитарних трансформација [9]:

$$|\psi_{\text{out}}\rangle = U_L \cdots U_2 U_1 |0\rangle^{\otimes n}.$$

Коначан одговор добија се мерењем. Овај модел је *универзалан*: теоријски може да реализује произвољан квантни алгоритам, укључујући Шоров алгоритам, Гроверову претрагу и варијационе алгоритме попут QAOA.

За ову дискусију нам је важна следећа интуиција: у моделу кола корисник не задаје директно енергетску функцију, већ конструише квантно коло које припрема, трансформише и мери стање. QAOA, на који ћемо касније осврнути, припада управо овој парадигми, али је концептуално блиско повезан са QUBO/Изинговим описом проблема.

Тренутни уређаји овог типа налазе се у *NISQ ери*: располажу ограниченим бројем шумовитих кубита и немају пуну корекцију грешака. Како дубина кола расте, расту и акумулација грешака, као и вероватноћа декохеренције, па је практично употребљива дубина кола ограничена. Због тога су у пракси најзанимљивији плитки хибридни алгоритми, нарочито варијациони приступи, уместо дубоких идеалних квантних кола.

2.3 Парадигма II: Квантно жарење (Quantum Annealing)

Квантно жарење је специјализована парадигма за оптимизационе проблеме [10]. Уместо општег квантног кола, улаз је енергетска функција чији минимум од-

говара решењу. Типично се полази од једноставног почетног Хамилтонијана

$$\hat{H}_0 = -\sum_{i=1}^{n} \hat{X}_i,$$

чије је основно стање лако за припрему, а затим се систем постепено води ка проблемском Хамилтонијану

$$\hat{H}_P = \sum_i h_i \hat{Z}_i + \sum_{i<j} J_{ij} \hat{Z}_i \hat{Z}_j,$$

који кодира циљну функцију.

Интерполација има облик

$$\hat{H}(t) = \left(1 - \frac{t}{T}\right) \hat{H}_0 + \frac{t}{T} \hat{H}_P, \qquad t \in [0, T].$$

Адијабатска теорема и спектрални процеп. Формална мотивација за овај приступ долази из адијабатске теореме: ако систем започне у основном стању Хамилтонијана $\hat{H}_0$ и ако се Хамилтонијан мења довољно споро, онда ће стање током еволуције остати близу тренутног основног стања [10]. Погодно је увести нормализовано време $s = t/T \in [0, 1]$ и писати

$$\hat{H}(s) = (1 - s)\hat{H}_0 + s\,\hat{H}_P.$$

Нека су $E_0(s) \leq E_1(s) \leq \cdots$ сопствене вредности $\hat{H}(s)$, а

$$\Delta(s) = E_1(s) - E_0(s)$$

тренутни енергетски процеп између основног и првог побуђеног стања. Тада је кључна количина

$$\Delta_{\min} = \min_{s \in [0,1]} \Delta(s),$$

јер што је минимални процеп мањи, то еволуција мора бити спорија да би се умањили прелази у побуђена стања. На хеуристичком нивоу често се пише услов облика

$$T \gg \frac{\max_s \left\| \frac{d\hat{H}}{ds} \right\|}{\Delta_{\min}^2},$$

који наглашава да сложеност адијабатске еволуције у великој мери зависи од најужег „уског грла"у спектру.

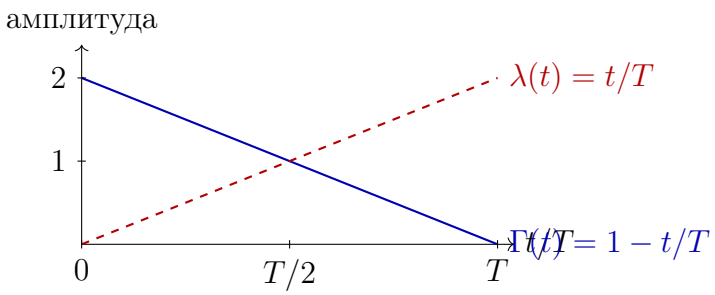
У контексту QUBO/Изингових проблема, $\hat{H}_P$ је дијагоналан у рачунарској бази, па свако базно стање $|z\rangle$ представља једну бинарну конфигурацију, док његова

енергија одговара вредности циљне функције. Ако систем на крају еволуције остане близу основног стања $\hat{H}_P$, мерење са великом вероватноћом враћа оптимално или бар нискоенергетско решење.

Интуитивно, квантно жарење замењује термално „прескакање"локалних минимума квантним тунелирањем кроз енергетске баријере. Управо зато је ова парадигма природно повезана са QUBO и Изинговим моделом, који енергетски опис проблема чине експлицитним.

Важно је, међутим, нагласити да је ово идеализован опис. Реални квантни анилери су отворени физички системи изложени шуму, декохеренцији, коначној температури и ограничењима представљања проблема на хардверу. Зато се у пракси адијабатска теорема пре користи као водећа интуиција него као строга гаранција да ће свака инстанца бити решена оптимално.

На слици 1 приказан је идеализовани распоред жарења.



Слика 1: Идеализовани распоред жарења: попречно поље $\Gamma(t)$ постепено опада, а проблемски члан $\lambda(t)$ расте. На крају еволуције доминира $\hat{H}_P$.

2.4 Поређење парадигми

За остатак рада довољно је запамтити једну кључну разлику [9, 10]: модел кола је *универзални модел рачунара*, док је квантно жарење *специјализовани модел за оптимизацију*. У нашем контексту то значи да квантни анилер директно користи QUBO/Изингову формулацију, док алгоритам QAOA користи кола, али над истом енергетском структуром.

У наставку се зато фокусирамо на један конкретан проблем (MaxCut), један класични референтни приступ (Z3), и један заједнички језик класичне и квантне оптимизације (QUBO).

3 Проблем MaxCut и НП-потпуност

Проблем MaxCut представља један од канонских НП-тешких проблема у теорији скожености израчунавања. Поред практичне важности у области VLSI

Табела 1: Поређење две главне квантне парадигме.

Особина	**Кола (Gate-Based)**	**Жарење (Annealing)**
Рачунски модел	Унитарна кола	Адијабатска еволуција
Улаз	Квантни програм	QUBO / Изингов модел
Примарна намена	Универзално рачунање	Комбинаторна оптимизација
Типичан излаз	Мерења након кола	Стање ниске енергије
Веза са QUBO	QAOA	D-Wave / анилерски модел

дизајна, статистичке физике и машинског учења, овај проблем служи и као природна полазна тачка за истраживање квантних оптимизационих алгоритама.

3.1 Класе сложености: P, NP, НП-тешко и НП-потпуно

Пре формалне дефиниције MaxCut-а, подсећамо на основне класе рачунарске сложености неопходне за разумевање тврђења која следе.

Дефиниција 3.1 (P [5])**.** *Класа P садржи све проблеме одлучивања решиве* детерминистичком Туринговом машином *у полиномском времену у односу на величину улаза n.*

Дефиниција 3.2 (NP [5])**.** *Класа NP садржи све проблеме одлучивања за које постоји* полиномски верификатор*: дат сертификат (сведок) c, провера да ли је c исправно решење за инстанцу x врши се у полиномском времену. Еквивалентно, NP је класа проблема решивих* недетерминистичком *Туринговом машином у полиномском времену.*

Дефиниција 3.3 (НП-тешки проблем [5])**.** *Проблем* Π *је* НП-тешки *ако сваки проблем из NP може бити полиномски редукован на* Π*:*

$$\forall L \in NP: \quad L \leq_p \Pi.$$

НП-тешки проблем је „бар тако тежак"као и сваки НП проблем, али не мора и сам припадати NP (нпр. може бити проблем оптимизације, а не одлучивања).

Дефиниција 3.4 (НП-потпуни проблем [5])**.** *Проблем* Π *је* НП-потпун *ако:*

1. $\Pi \in NP$ *(решења се верификују у полиномском времену), и*
2. Π *је НП-тешки (сваки НП проблем се полиномски редукује на* Π*).*

НП-потпуни проблеми су „најтежи"у класи NP.

Напомена. Централно отворено питање теорије сложености је да ли важи $\mathsf{P} = \mathsf{NP}$. Под претпоставком $\mathsf{P} \neq \mathsf{NP}$, ниједан НП-тешки проблем нема полиномски алгоритам, па ни MaxCut.

3.2 Формална дефиниција

Дефиниција 3.5 (Рез и MaxCut [7]). *Нека је $G = (V, E, w)$ неусмерени граф са тежинама $w : E \to \mathbb{R}_+$. Рез је партиција $V = S \cup \bar{S}$, $S \cap \bar{S} = \emptyset$. Тежина реза дефинише се као*

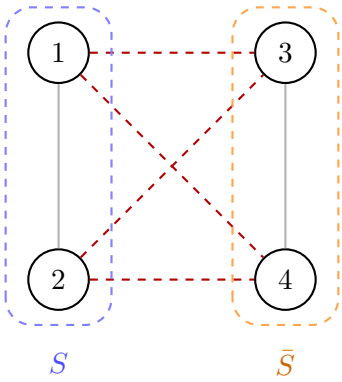
$$\mathrm{cut}(S, \bar{S}) \;=\; \sum_{\substack{\{u,v\} \in E \\ u \in S,\, v \in \bar{S}}} w(u, v).$$

Проблем MaxCut *тражи партицију која максимизује* $\mathrm{cut}(S, \bar{S})$ *преко свих могућих избора $S \subseteq V$.*

У верзији одлучивања, проблем гласи: *да ли постоји рез тежине $\geq k$?* — управо ова верзија је НП-потпуна.

3.3 Илустративни пример

Посматрајмо потпуни граф K_4 са јединичним тежинама. Оптималан рез добијамо поделом чворова у два пара: $S = \{1, 2\}$, $\bar{S} = \{3, 4\}$. Ивице које прелазе рез (обојене и испрекидане у доњем дијаграму) јесу $\{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}$, па је $\mathrm{OPT}(K_4) = 4$. Ивице унутар партиција ($\{1, 2\}$ и $\{3, 4\}$) не улазе у рез.



Слика 2: Оптималан рез у K_4: испрекидане ивице прелазе рез и доприносе $\mathrm{cut}(S, \bar{S}) = 4 = \mathrm{OPT}$.

3.4 НП-потпуност проблема MaxCut

Теорема 3.6 (Карп, 1972 [4]). *Проблем* MaxCut *(у верзији одлучивања) је NP-потпун.*

Напомена. Горња редукција важи за *непондерисани* MaxCut. Тежински MaxCut такође је НП-тешки (и НП-потпун у верзији одлучивања) за рационалне тежине, уз стандардну нумеричку аргументацију.

3.5 Апроксимациона сложеност

НП-тешкоћа онемогућава постојање полиномског егзактног алгоритма (уз стандардну претпоставку $\mathsf{P} \neq \mathsf{NP}$), па је природно питање колико добро можемо апроксимирати оптимум.

Дефиниција 3.7 (Апроксимациони однос [7])**.** *Алгоритам $\mathcal{A}$ остварује* апроксимациони однос $\alpha \leq 1$ *ако за сваку инстанцу G важи* $\mathrm{cut}(\mathcal{A}(G)) \geq \alpha \cdot \mathrm{OPT}(G)$.

Тривијалан рандомизовани алгоритам — сврсти сваки чвор насумице у S или $\bar{S}$ — остварује $\alpha = 1/2$ у очекивању, јер свака ивица прелази рез са вероватноћом $1/2$.

Гоеманс и Вилијамсон [3] предложили су алгоритам заснован на семидефинитном програмирању (SDP) и хиперраванској рандомизацији са знатно бољом гаранцијом:

$$\mathrm{cut}(S_{\mathrm{alg}}) \;\geq\; \alpha_{\mathrm{GW}} \cdot \mathrm{OPT}, \qquad \alpha_{\mathrm{GW}} = \min_{0 \leq \theta \leq \pi} \frac{2}{\pi} \cdot \frac{\theta}{1 - \cos\theta} \;\approx\; 0{,}878.$$

Напомена. Под *Хипотезом јединствених игара* (Unique Games Conjecture, Khot 2002), Гоеманс-Вилијамсонова константа $\alpha_{\mathrm{GW}} \approx 0{,}878$ представља *оптималну* полиномски достижну апроксимацију. Ово даје теоријску мотивацију за истраживање суперполиномских приступа, укључујући квантне алгоритме попут QUBO са квантним жарачима и QAOA.

4 Класични приступ: Z3 решавач и SMT

Пре него што пређемо на квантне приступе, важно је разумети домет савремених класичних алата за решавање НП-тешких проблема. SMT решавачи, са Z3 као репрезентативним примером, представљају врхунац класичне аутоматске провере задовољивости и оптимизације малих до средње великих инстанци.

4.1 SAT и SMT решавачи

Стандардни модерни приступ решавању комбинаторних проблема у аутоматском резоновању ослања се на два хијерархијски повезана формализма: SAT (булова задовољивост) као основни слој и SMT (задовољивост по теоријама) као његово богатије проширење. У наставку описујемо оба.

SAT решавачи. Проблем булове задовољивости (SAT) пита: постоји ли вредновање булових променљивих које задовољава дату исказну формулу у конјунктивној нормалној форми (КНФ)? SAT је НП-потпун, али савремени решавачи засновани на *DPLL* алгоритму са *CDCL* (Conflict-Driven Clause Learning) проширењем [8] решавају инстанце са милионима клауза у пракси.

SMT решавачи. *Satisfiability Modulo Theories* (SMT) проширује SAT додатним теоријама над конкретним доменима: линеарном аритметиком, низовима, битвекторима, непроширеним функцијама итд. [8] Решавач комбинује CDCL SAT „језгро"са специјализованим *теоријским решавачима* за сваку теорију.

4.2 Z3: Архитектура и могућности

Z3 је SMT решавач развијен у Microsoft Research [1, 2] (de Moura и Bjørner, 2008). Подржава следеће теорије и конструкте:

- линеарна и нелинеарна аритметика над $\mathbb{Z}$ и $\mathbb{R}$,
- битвектори фиксне ширине,
- низови и скупови,
- неинтерпретиране функције и еквалитет,
- квантификатори (са ограниченом потпуношћу).

Осим провере задовољивости, Z3 подржава и оптимизацију преко `Optimize` пакета: корисник задаје циљну функцију коју решавач максимизује или минимизује уз дата ограничења. Интерно се користи νZ радни оквир, који комбинује Паретово оптимизовање, лексикографске циљеве и MaxSMT стратегије у јединствену архитектуру.

Python API (`z3-solver`) чини Z3 доступним за брзо прототиповање. Сада ћемо видети како се MaxCut проблем кодира у Z3.

4.3 Кодирање MaxCut у Z3

За граф $G = (V, E)$ са $|V| = n$ чворова уводимо булове (или целобројне) индикаторске променљиве:

$$x_v \in \{0, 1\}, \quad v \in V,$$

где $x_v = 1$ означава $v \in S$, а $x_v = 0$ означава $v \in \bar{S}$. Ивица $\{u, v\}$ прелази рез ако и само ако $x_u \neq x_v$, тј. $x_u \oplus x_v = 1$. Циљна функција коју максимизујемо је:

$$\mathrm{cut}(x) = \sum_{\{u,v\} \in E} \big(x_u \oplus x_v\big) = \sum_{\{u,v\} \in E} \big(x_u + x_v - 2x_u x_v\big).$$

Пример 4.1 (Егзактни MaxCut за K_4 у Z3 (Python)).

```python
from z3 import *

# Indikatorske promenljive: x[v] in {0, 1}
x = [Int(f'x{i}') for i in range(4)]
opt = Optimize()
for v in x:
    opt.add(Or(v == 0, v == 1))

# Ivice potpunog grafa K_4
edges = [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]

# Tezina reza: ivica (u,v) doprinosi 1 ako x[u] != x[v]
cut = Sum([If(x[u] != x[v], 1, 0) for u, v in edges])
h = opt.maximize(cut)

if opt.check() == sat:
    m = opt.model()
    vals = [m[v].as_long() for v in x]
    print(f"Optimum: {opt.upper(h)}")   # 4
    print(f"Particija: {vals}")          # npr. [0, 0, 1, 1]
```

Листинг 1: MaxCut на K_4 помоћу Z3 Optimize пакета.

Z3 налазе оптималну партицију $S = \{0, 1\}$, $\bar{S} = \{2, 3\}$ са вредношћу реза $4 = \mathrm{OPT}(K_4)$, у складу са анализом из поглавља 3 (Слика 2).

Повезаност са 3-SAT кодирањем. MaxCut се такође може кодирати директно у КНФ и предати SAT решавачу. За сваку могућу тежину реза k dodajemo поређење $\mathrm{cut}(x) \geq k$ и бинарним претражевањем налазимо оптимум. Иако је ово функционално тачно, `Optimize` библиотека у Z3 је директније и типично брже за целобројне циљне функције.

4.4 Ограничења SMT/SAT приступа

Теоријска баријера. Иако је Z3 изузетно моћан алат, НП-тешкоћа проблема MaxCut гарантује да у најгорем случају не постоји полиномски алгоритам (под претпоставком $\mathsf{P} \neq \mathsf{NP}$). Број могућих партиција је 2^n, а CDCL претрага у пракси показује *експоненцијално* расте времена у зависности од структуре инстанце [8].

Напомена. Z3 и сродни SMT решавачи служе двострукој улози у овом раду: као *референтни алати* за верификацију тачности других приступа на малим

инстанцама, и као полазна тачка за поређење са квантним QUBO жарењем (поглавље 6).

5 QUBO: Quadratic Unconstrained Binary Optimization

Пре него што пређемо на квантне алгоритме, неопходно је увести *QUBO* формулацију. QUBO је стандардни математички оквир у коме се комбинаторни оптимизациони проблеми изражавају на начин директно погодан за квантне и квантно-инспирисане решаваче.

5.1 Дефиниција и основна структура

Дефиниција 5.1 (QUBO [6])**.** Проблем квадратне неограничене бинарне оптимизације *(Quadratic Unconstrained Binary Optimization, QUBO) тражи бинарни вектор* $\mathbf{x} \in \{0,1\}^n$ *који минимизује (или максимизује) квадратну форму:*

$$f(\mathbf{x}) = \mathbf{x}^\top Q\,\mathbf{x} = \sum_{i=1}^{n} Q_{ii}\,x_i + 2\sum_{i<j} Q_{ij}\,x_i x_j,$$

где је $Q \in \mathbb{R}^{n\times n}$ *симетрична (или горња троугаона) матрица коефицијената.*

Пример 5.2 (QUBO матрица за $n = 3$)**.** *Нека је* $n = 3$ *и*

$$Q = \begin{pmatrix} -1 & 1 & 0 \\ 1 & -1 & 1 \\ 0 & 1 & -1 \end{pmatrix}.$$

Тада је циљна функција

$$f(\mathbf{x}) = -x_1 - x_2 - x_3 + 2x_1x_2 + 2x_2x_3.$$

За $\mathbf{x} = (1,0,1)^\top$ *добијамо* $f = -1-0-1+0+0 = -2$*, а за* $\mathbf{x} = (1,1,1)^\top$ *добијамо* $f = -1-1-1+2+2 = 1$*. Минимум је дакле у* $\mathbf{x}^* = (1,0,1)^\top$ *са вредношћу* $f^* = -2$*.*

Три кључне особине чине QUBO централним форматом:

- **Бинарност:** $x_i^2 = x_i$ за $x_i \in \{0,1\}$, па дијагонални чланови $Q_{ii}x_i^2$ редукују на линеарне чланове $Q_{ii}x_i$.
- **Без ограничења:** Сва ограничења се уграђују у циљну функцију преко казнених чланова (*penalty terms*).
- **Квантна еквиваленција:** QUBO директно одговара *Изинговом* (Ising) спин моделу, природном Хамилтонијану квантних анилера и QAOA алго-

ритма [10].

5.2 Примери QUBO формулација ограничења

Конвертовање ограничених проблема у QUBO захтева да свако ограничење заменимо са казненим чланом пропорционалним са $\lambda > 0$ (довољно великим да казни свако кршење услова) [6]:

Ограничење бинарности $x_i + x_j = 1$ (тачно један од два бита је 1):

$$P_1 = \lambda\,(x_i + x_j - 1)^2 = \lambda\,(x_i + x_j - 2x_ix_j - 1 + 2x_ix_j) = \lambda\,(1 - x_i - x_j + 2x_ix_j - 1 + ...)$$

Развијањем добијамо:

$$P_1 = \lambda\,\big(1 - x_i - x_j + 2x_ix_j\big).$$

У QUBO матрици то одговара: $Q_{ii} -= \lambda$, $Q_{jj} -= \lambda$, $Q_{ij} += 2\lambda$.

Ограничење неједнакости $x_i + x_j \leq 1$ (не могу оба бити 1 истовремено):

$$P_2 = \lambda\, x_ix_j.$$

Кршење (оба = 1) додаје λ циљној функцији.

Тачно-k-од-n ограничење $\sum_{i=1}^{n} x_i = k$:

$$P_3 = \lambda\,\Big(\sum_{i=1}^{n} x_i - k\Big)^2 = \lambda\,\Big(k^2 - 2k\sum_i x_i + \sum_i x_i^2 + 2\sum_{i<j} x_ix_j\Big).$$

Пошто $x_i^2 = x_i$:

$$P_3 = \lambda\,\Big((1-2k)\sum_i x_i + 2\sum_{i<j} x_ix_j + k^2\Big).$$

Пример 5.3 (3-бојење чворова — QUBO казна)**.** *За ограничење да суседни чворови u и v у графу не смеју добити исту боју c уводимо бинарне варијабле $x_{u,c} \in \{0,1\}$. Ограничење „u и v не деле боју c"кодира се казном:*

$$P_{u,v,c} = \lambda\, x_{u,c}\, x_{v,c}.$$

Свако кршење (оба чвора добијају боју c) повећава вредност функције за λ, чиме решавач избегава та стања.

Табела 2 сажима најчешће типове ограничења и одговарајуће QUBO казнене чланове.

Табела 2: Стандардна ограничења и њихове QUBO казне ($\lambda > 0$).

Ограничење	**Казнени члан** P	**Значење**
$x_i = 0$	$\lambda\, x_i$	Искључи чвор/ресурс из решења
$x_i = 1$	$\lambda\,(1 - x_i)$	Обавезно укључи у решење
$x_i + x_j = 1$	$\lambda\,(1 - x_i - x_j + 2x_ix_j)$	Тачно jedan од два бита активан
$x_i + x_j \leq 1$	$\lambda\, x_ix_j$	Не могу оба бити активна истовремено
$x_i = x_j$	$\lambda\,(x_i + x_j - 2x_ix_j)$	Оба бита морају имати исту вредност
$x_i \leq x_j$	$\lambda\,(x_i - x_ix_j)$	Ако је x_i активан, мора бити и x_j
$\sum_{i=1}^{n} x_i = k$	$\lambda\,\left(\sum_{i=1}^{n} x_i - k\right)^2$	Тачно k битова активно

5.3 Пример примене QUBO казни: тачно-2-од-3 ограничење

Нека су дате три бинарне променљиве $x_1, x_2, x_3 \in \{0, 1\}$ и линеарна циљна функција:

$$f_{\text{obj}}(\mathbf{x}) = -4x_1 - 3x_2 - 5x_3.$$

Желимо да минимизујемо f_{obj} уз ограничење да тачно две од три променљиве морају бити активне:

$$x_1 + x_2 + x_3 = 2.$$

Казнени члан. Из табеле 2, ограничење $\sum_i x_i = k$ кодира се казном $\lambda\,(\sum_i x_i - k)^2$. За $k = 2$ и $\lambda = 10$ добијамо:

$$P(\mathbf{x}) = 10\,(x_1 + x_2 + x_3 - 2)^2.$$

Потпуна QUBO формулација.

$$f(\mathbf{x}) = f_{\text{obj}}(\mathbf{x}) + P(\mathbf{x}) = -4x_1 - 3x_2 - 5x_3 + 10\,(x_1 + x_2 + x_3 - 2)^2.$$

Провера на свим изводљивим решењима (тачно два бита активна):

$\mathbf{x}$	f_{obj}	P	f
$(1, 1, 0)$	-7	0	-7
$(1, 0, 1)$	-9	0	$\mathbf{-9}$
$(0, 1, 1)$	-8	0	-8

За неизводљиво решење, нпр. $\mathbf{x} = (1, 1, 1)$:

$$f = -12 + 10 \cdot (3 - 2)^2 = -12 + 10 = -2,$$

што је лошије од сваког изводљивог решења, па решавач природно бира $\mathbf{x}^* = (1, 0, 1)$ са вредношћу $f^* = -9$.

5.4 Аналогија са Лагранжевим множитељима

Структура QUBO формулације са казненим члановима подсећа на класичну методу *Лагранжевих множитеља* из математичке оптимизације. У континуалном случају, проблем

$$\min_{\mathbf{x}} \; f(\mathbf{x}) \quad \text{уз} \quad g_j(\mathbf{x}) = 0, \; j = 1, \ldots, m$$

решава се увођењем Лагранжијана:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{j=1}^{m} \lambda_j \, g_j(\mathbf{x}),$$

где су λ_j Лагранжеви множитељи. Оптимум се тражи у стационарним тачкама $\nabla_{\mathbf{x}} \mathcal{L} = 0$.

QUBO приступ са казненим члановима прати исту идеју, само прилагођену бинарном, дискретном домену: уместо линеарних члanova $\lambda_j g_j(\mathbf{x})$ користе се *квадратне казне* $\lambda_j P_j(\mathbf{x}) \geq 0$ које су нула ако и само ако је ограничење задовољено:

$$f_{\text{QUBO}}(\mathbf{x}) = f(\mathbf{x}) + \sum_{j=1}^{m} \lambda_j \, P_j(\mathbf{x}), \qquad \mathbf{x} \in \{0, 1\}^n.$$

Кључна разлика је у томе што се у Лагранжевој методи λ_j одређују тражењем тачке превоја, а у QUBO приступу бирају *унапред* довољно великим да обезбеде да ниједно кршење не може бити енергетски повољније од легалног решења.

5.5 Формулација MaxCut-а као QUBO

Вратимо се проблему MaxCut из поглавља 3 [6]. Нека је $G = (V, E, w)$ граф са тежинама, $|V| = n$.

Варијабле. Уведемо бинарну индикаторску променљиву за сваки чвор:

$$x_v \in \{0, 1\}, \quad v \in V,$$

где $x_v = 0$ значи $v \in S$, а $x_v = 1$ значи $v \in \bar{S}$.

Циљна функција. Ивица $\{u, v\}$ прелази рез ако и само ако $x_u \neq x_v$, тј. $x_u + x_v - 2x_u x_v = 1$. Тежина реза коју *максимизујемо* је:

$$\mathrm{cut}(\mathbf{x}) = \sum_{\{u,v\} \in E} w_{uv} \, (x_u + x_v - 2x_u x_v).$$

Конверзија у QUBO минимизацију. Пошто QUBO стандардно тражи минимум, негирамо циљ:

$$\boxed{f_{\mathrm{MaxCut}}(\mathbf{x}) = -\sum_{\{u,v\} \in E} w_{uv} \, (x_u + x_v - 2x_u x_v).}$$

Матрица Q. Елементи QUBO матрице читају се директно из горњег израза:

$$Q_{vv} = -\sum_{u:\{u,v\} \in E} w_{uv} \quad \text{(дијагонала)}, \qquad Q_{uv} = w_{uv} \quad \text{(вандијагонала, } u \neq v\text{)}.$$

Нема додатних ограничења. Проблем је *природно неограничен*, па не треба казнени чланови.

Пример 5.4 (QUBO матрица за K_4 са јединичним тежинама)**.** *За K_4 ($n = 4$, свих 6 ивица са $w = 1$):*

$$Q = \begin{pmatrix} -3 & 1 & 1 & 1 \\ 1 & -3 & 1 & 1 \\ 1 & 1 & -3 & 1 \\ 1 & 1 & 1 & -3 \end{pmatrix}$$

($Q_{vv} = -(n-1) = -3$ за сваки чвор, $Q_{uv} = Q_{vu} = 1$ за сваку ивицу).

Оптималан бинарни вектор, нпр. $\mathbf{x} = (0, 0, 1, 1)$ *($S = \{0, 1\}$, $\bar{S} = \{2, 3\}$), даје:*

$$f(\mathbf{x}) = -4 = -\mathrm{OPT}(K_4),$$

у складу са резултатом из поглавља 3.

Напомена. QUBO је „заједнички језик"класичних и квантних оптимизационих решавача: иста матрица Q може бити прослеђен D-Wave анилеру, класичном симулираном анилеру или QAOA алгоритму, без икакве измене формулације проблема [10].

5.6 Решавање QUBO: Z3 и симулирано жарење

Да бисмо илустровали портабилност QUBO формата, решавамо исту инстанцу MaxCut на K_4 са QUBO матрицом из претходног примера. Решавамо потпуно различитим алатима: егзактним SMT решавачем Z3 и класичним симулираним жарењем преко D-Wave Ocean SDK-а.

5.6.1 Решавање QUBO помоћу Z3

Z3-ов `Optimize` пакет директно прима квадратну циљну функцију над целобројним (или буловим) променљивама. Бинарност се наноси као скуп ограничења $x_i \in \{0, 1\}$, а решавач налазе егзактни минимум применом νZ оптимизационог оквира.

Пример 5.5 (QUBO за MaxCut на K_4 помоћу Z3 (Python)).

```python
from z3 import *

# QUBO matrica za MaxCut na K4 (simetricna forma)
Q = {
    (0,0): -3, (1,1): -3, (2,2): -3, (3,3): -3,
    (0,1):  1, (0,2):  1, (0,3):  1,
    (1,0):  1, (1,2):  1, (1,3):  1,
    (2,0):  1, (2,1):  1, (2,3):  1,
    (3,0):  1, (3,1):  1, (3,2):  1,
}

n = 4
x = [Int(f'x{i}') for i in range(n)]
opt = Optimize()

# Binarne promenljive: x[i] in {0, 1}
for v in x:
    opt.add(Or(v == 0, v == 1))

# QUBO ciljna funkcija: sum_{i,j} Q[i,j] * x[i] * x[j]
obj = Sum([
    Q.get((i, j), 0) * x[i] * x[j]
    for i in range(n) for j in range(n)
])
opt.minimize(obj)

if opt.check() == sat:
    m = opt.model()
    vals = [m[v].as_long() for v in x]
```

```
    print(f"Minimalna vrednost: {m.eval(obj)}")  # -4
    print(f"Resenje x: {vals}")                  # [1, 1, 0,
        0]
```

Листинг 2: Решавање QUBO помоћу Z3 Optimize пакета.

Z3 проналази оптималан вектор $\mathbf{x} = (0, 0, 1, 1)$ са вредношћу $f(\mathbf{x}) = -4$, у складу са аналитичким резултатом. Решавач гарантује глобалну оптималност, али временска сложеност у најгорем случају остаје експоненцијална.

Инверзни проблем: реконструкција матрице Q из познатог решења. Сада разматрамо обрнут смер: претпоставимо да је познато да је $\mathbf{x}^* = (0, 0, 1, 1)$ *оптимално* решење неког QUBO проблема, али матрица Q није позната. Циљ је употребити Z3 да нађе Q такву да $\mathbf{x}^*$ минимизује $f(\mathbf{x}) = \mathbf{x}^\top Q\,\mathbf{x}$ над свим $\mathbf{x} \in \{0, 1\}^n$.

Услов оптималности. Вектор $\mathbf{x}^*$ је глобални минимум ако и само ако:

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall\, \mathbf{x} \in \{0, 1\}^n.$$

За фиксно $\mathbf{x}^*$ и фиксно $\mathbf{x}$, ова неједнакост је *линеарна* у елементима матрице Q, стога је проблем налажења Q систем линеарних ограничења, природан за Z3.

Неодређеност. Систем са 2^n ограничења и $n(n+1)/2$ слободних улаза у Q је генерално недоређен: много различитих матрица Q може имати исти оптимум. Додатна нормализациона ограничења (нпр. фиксирање скале или структуре) отклањају ту вишезначност.

Пример 5.6 (Реконструкција могућих Q за коју је $(0, 0, 1, 1)$ оптимум (Python))**.**

```
from z3 import *
from itertools import product

n      = 4
x_star = [0, 0, 1, 1]   # poznato optimalno resenje

# Slobodne (cjelobrojne) promenljive za simetricnu Q matricu
Q = {(i, j): Int(f'Q_{i}_{j}')
     for i in range(n) for j in range(n)}

s = Solver()

# Simetricnost: Q[i,j] == Q[j,i]
for i in range(n):
```

```
    for j in range(n):
        s.add(Q[(i, j)] == Q[(j, i)])

# Ogranicenje domene da pretraga bude konacna
for i in range(n):
    for j in range(i, n):
        s.add(Q[(i, j)] >= -10)
        s.add(Q[(i, j)] <=  10)

def energy(x):
    """f(x) = sum_{i,j} Q[i,j] * x[i] * x[j]"""
    return Sum([Q[(i, j)] * x[i] * x[j]
                for i in range(n) for j in range(n)])

# Uslov optimalnosti: f(x*) <= f(x) za svaki x in {0,1}^n
f_star = energy(x_star)
for x in product([0, 1], repeat=n):
    s.add(f_star <= energy(list(x)))

# Enumeracija prvih 100 razlicitih simetricnih Q matrica
count = 0
while count < 100 and s.check() == sat:
    m  = s.model()
    count += 1
    vals = {(i, j): m.eval(Q[(i, j)]).as_long()
            for i in range(n) for j in range(i, n)}
    print(f"\nMatrica {count}:")
    for i in range(n):
        print([vals.get((min(i,j), max(i,j))) for j in range(n
            )])
    # Blokiranje trenutnog resenja
    s.add(Or([Q[(i, j)] != vals[(i, j)]
              for i in range(n) for j in range(i, n)]))

print(f"\nUkupno pronadjeno: {count} matrica")
```

Листинг 3: Z3 реконструкција QUBO матрице из познатог оптималног решења.

Z3 проналази могућа решења под датим условима ограничења. Свака пронађена матрица Q задовољава услов оптималности за $\mathbf{x}^*$, али може имати различите вредности. Тиме се добијају QUBO матрице које би потенцијално одговарале K_4 MaxCut проблему.

5.6.2 Решавање QUBO симулираним жарењем (D-Wave Ocean SDK)

D-Wave Ocean SDK садржи класични `SimulatedAnnealingSampler` који прима исти QUBO формат и враћа скуп узорака сортираних по енергији [13]. Самплер је квантно-инспирисан: не захтева стварни квантни хардвер, али прати исту конвенцију интерфејса као и правилни D-Wave анилер [11], тако да је прелаз на хардвер тривијалан (замена самплера са `DWaveSampler`).

Пример 5.7 (QUBO за MaxCut на K_4 помоћу SimulatedAnnealingSampler (Python)).

```python
from dwave.samplers import SimulatedAnnealingSampler

# Simetricna QUBO matrica za MaxCut na K4
Q = {
    (0,0): -3, (0,1):  1, (0,2):  1, (0,3):  1,
    (1,0):  1, (1,1): -3, (1,2):  1, (1,3):  1,
    (2,0):  1, (2,1):  1, (2,2): -3, (2,3):  1,
    (3,0):  1, (3,1):  1, (3,2):  1, (3,3): -3,
}

sampler = SimulatedAnnealingSampler()
# num_reads: broj nezavisnih pokretanja zarenja
response = sampler.sample_qubo(Q, num_reads=100)

best = response.first
print(f"Energija: {best.energy}")        # -4.0
print(f"Resenje: {dict(best.sample)}")   # {0:0, 1:0, 2:1, 3:1}
```

Листинг 4: Решавање QUBO симулираним жарењем преко D-Wave Ocean SDK-а.

Метод `sample_qubo` покреће `num_reads` независних трагова жарења и враћа резултате у облику `SampleSet` објекта. Атрибут `response.first` садржи узорак са најнижом енергијом. Такође треба обратити пажњу да постоји више оптималиних решења (нпр. $(0, 0, 1, 1)$ и $(1, 1, 0, 0)$), па се може добити било које од њих. За прелаз на стварни D-Wave хардвер довољно је заменити самплер:

```python
from dwave.system import DWaveSampler, EmbeddingComposite

sampler = EmbeddingComposite(DWaveSampler())
response = sampler.sample_qubo(Q, num_reads=100)
```

Листинг 5: Прелаз на D-Wave хардвер тј. само се самплер мења.

`EmbeddingComposite` аутоматски проналази уграђивање логичких QUBO чворова у физички граф процесора, скривајући детаље архитектуре од корисника.

Напомена. Оба приступа прихватају идентичан QUBO формат Q. Z3 гарантује егзактно решење, али не скалира на велике инстанце. Симулирано жарење је хеуристичко и не гарантује оптималност, али лако обрађује инстанце са стотинама варијабли. Квантни анилер (D-Wave) нуди исти интерфејс уз потенцијалну предност тунелирања на хардверском нивоу [10].

6 QAOA и MaxCut

QAOA (*Quantum Approximate Optimization Algorithm*) предложили су Farhi, Goldstone и Gutmann [12] као хибридни квантно-класични алгоритам за комбинаторну оптимизацију. За наше потребе није пресудан детаљан профил перформанси QAOA, већ чињеница да он показује како се *иста* проблемска формулација, добијена у облику QUBO/Изинговог модела, може користити и у моделу квантних кола.

Веза QUBO → Изинг → QAOA. Сваки QUBO проблем се супституцијом $x_i = (1 - s_i)/2$ директно пресликава у Изингов Хамилтонијан облика $H = \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j$, где спинови $s_i \in \{-1, +1\}$ одговарају кубитима у стањима $|0\rangle$ и $|1\rangle$. QAOA затим узима тај Хамилтонијан *директно* као проблемски оператор $\hat{H}_C$ без икакве додатне трансформације. Другим речима, QUBO матрица Q потпуно одређује квантно коло QAOA-е: дијагонални елементи Q_{ii} дају локална поља h_i, а вандијагонални Q_{ij} дају спрезања J_{ij}.

6.1 Од MaxCut-а до проблемског Хамилтонијана

За MaxCut уводимо спин варијабле $z_v \in \{-1, +1\}$ за сваки чвор $v \in V$. Ивица $\{u, v\}$ прелази рез ако и само ако $z_u z_v = -1$, па је величина реза

$$C(\mathbf{z}) = \sum_{\{u,v\} \in E} \frac{1 - z_u z_v}{2}.$$

Ова циљна функција директно прелази у проблемски Хамилтонијан

$$\hat{H}_C = \frac{1}{2} \sum_{\{u,v\} \in E} \left(\hat{I} - \hat{Z}_u \hat{Z}_v\right).$$

Кључна ствар је да је то *иста* енергетска структура коју смо већ добили из QUBO формулације у поглављу 5. Зато QAOA није нова формулација проблема, већ нови начин да се оптимизује већ познати Хамилтонијан.

6.2 Зашто је QAOA важан

У контексту наше дискусије, QAOA је важан из три разлога.

- Показује да се MaxCut може решавати у моделу квантних кола над истим QUBO/Изинговим описом који користе квантни анилери.
- Илуструје хибридни образац: квантни део претражује простор стања, а класични део управља оптимизацијом.
- Пружа природно место за поређење са Z3: QAOA тражи добро решење варијационо, док Z3 може егзактно да провери мале инстанце и исправност добијеног bitstring-а.

Практично ограничење QAOA у NISQ окружењу јесте то што дубина кола, шум и цена класичне оптимизационе петље брзо постају доминантни. Зато је у овом раду QAOA пре свега *концептуални мост* између QUBO формулације и квантног рачунарства заснованог на колима и указујемо на њега као пример, како се исти проблем може решавати у различитим моделима рачунања.

7 Квантно рачунарство и аутоматско резоновање

7.1 Z3 као верификатор QUBO ограничења

Кључна слабост QUBO формулације је у томе што су ограничења уграђена *индиректно*, преко казнених чланова. Ако је знак погрешан, ако недостаје један квадратни члан, или ако је коефицијент казне сувише мали, минимум енергије више не мора да одговара важећем решењу оригиналног проблема. Z3 је зато користан не само као решавач, већ и као *верификатор исправности саме QUBO трансформације*.

Нека је оригинални проблем дат у облику

$$\min g(\mathbf{x}) \qquad \text{уз услове} \qquad C_1(\mathbf{x}), \ldots, C_m(\mathbf{x}),$$

а QUBO апроксимација као

$$f(\mathbf{x}) = g(\mathbf{x}) + \sum_{j=1}^{m} \lambda_j P_j(\mathbf{x}), \qquad \mathbf{x} \in \{0,1\}^n.$$

Тада Z3 може да послужи у најмање три различита корака верификације.

1. Провера семантике казнених члана. За свако ограничење треба доказати да казнени члан заиста разликује дозвољена и недозвољена стања:

$$C_j(\mathbf{x}) \iff P_j(\mathbf{x}) = 0, \qquad \neg C_j(\mathbf{x}) \Rightarrow P_j(\mathbf{x}) > 0.$$

На пример, за ограничење „тачно један"$x_i + x_j = 1$, penalty члан

$$P(\mathbf{x}) = \lambda\,(1 - x_i - x_j + 2x_i x_j)$$

је исправан ако Z3 покаже да је формула

$$x_i, x_j \in \{0, 1\} \;\wedge\; \big((x_i + x_j = 1 \wedge P \neq 0) \vee (x_i + x_j \neq 1 \wedge P = 0)\big)$$

незадовољива. На тај начин добијамо *контрапример ако кодирање није добро*, уместо да се ослањамо само на ручни алгебарски развој.

Пример 7.1 (Провера казненог члана помоћу Z3)**.**

```
from z3 import *

xi, xj, lam = Ints('xi xj lam')

# Kazneni clan za ogranicenje "tacno jedan":
P = lam * (1 - xi - xj + 2 * xi * xj)

s = Solver()
s.add(Or(xi == 0, xi == 1))
s.add(Or(xj == 0, xj == 1))
s.add(lam > 0)

# Trazenje kontraprimera:
# validne vrednosti su one gde je P=0, a nevalidne gde je P>0
s.add(Or(
    And(xi + xj == 1, P != 0),
    And(xi + xj != 1, P <= 0)
))

print(s.check())  # unsat
```

Листинг 6: Z3 проверава да казнени члан добро кодира ограничење $x_i + x_j = 1$.

Резултат `unsat` значи да контрапример не постоји: казнени члан је нула тачно на дозвољеним стањима, а строго позитиван на недозвољеним.

2. Провера да је казна довољно велика. Чак и ако је $P_j(\mathbf{x}) = 0$ тачно на дозвољеним стањима, коефицијент λ_j мора бити довољно велик да ниједно недозвољено стање не постане енергетски повољније од исправног решења. За мале и средње инстанце Z3 може прво да нађе оптимално дозвољено решење

$\mathbf{x}^\star$, а затим да провери да ли постоји контрапример облика

$$\exists\, \mathbf{x} \in \{0,1\}^n : \neg C(\mathbf{x}) \wedge f(\mathbf{x}) \leq f(\mathbf{x}^\star).$$

Ако је ова формула `unsat`, онда ниједно кршење ограничења не може да „пробије"легално решење у QUBO енергији. То је формални доказ да је изабрана казнена скала довољна.

Пример 7.2 (Провера да је казнени коефицијент довољно велик)**.**

```
from z3 import *

# Originalni problem ogranizenja:
# maksimizuj 3*x0 + 2*x1  sa uslovom x0 + x1 <= 1
x0, x1 = Ints('x0 x1')

opt = Optimize()
opt.add(Or(x0 == 0, x0 == 1), Or(x1 == 0, x1 == 1))
opt.add(x0 + x1 <= 1)

reward = 3 * x0 + 2 * x1
h = opt.maximize(reward)

assert opt.check() == sat
m = opt.model()
best_reward = m.eval(reward).as_long()   # 3
best_energy = -best_reward               # -3

def violates_feasible_optimum(lam_value):
    y0, y1 = Ints('y0 y1')
    s = Solver()
    s.add(Or(y0 == 0, y0 == 1), Or(y1 == 0, y1 == 1))

    qubo = -(3 * y0 + 2 * y1) + lam_value * y0 * y1

    # Potrazi nedozvoljeno resenje koje je bar jednako dobro
       kao i najbolje dozvoljeno
    s.add(y0 + y1 > 1)
    s.add(qubo <= best_energy)

    return s.check()

print(violates_feasible_optimum(2))  # sat
print(violates_feasible_optimum(3))  # unsat
```

Листинг 7: Z3 провера да је казнени коефицијент λ довољно велик.

Овде `sat` за $\lambda = 2$ значи да и даље постоји недозвољено стање које достиже енергију најбољег легалног решења, док `unsat` за $\lambda = 3$ формално потврђује да је та казна довољна.

3. Верификација резултата квантног решавача. Када QAOA или квантни анилер врате кандидатно решење $\hat{\mathbf{x}}$, Z3 може независно да провери:

- да ли $\hat{\mathbf{x}}$ задовољава оригинална ограничења,
- колика је вредност оригиналне циљне функције $g(\hat{\mathbf{x}})$,
- да ли постоји боље легално решење за исту малу инстанцу.

Тако Z3 постаје алат за процену квалитета квантних и хеуристичких метода: квантни алгоритам тражи ниску енергију, а SMT верификатор потврђује да та ниска енергија има жељено семантичко значење.

Пример 7.3 (Верификација кандидатног bitstring-а)**.**

```
from z3 import *

# Kandidat koji je vratio aniler ili QAOA
x_hat = [0, 1, 0]
weights = [5, 4, 1]

# Originalni problem: maksimizuj 5*x0 + 4*x1 + x2
# sa ogranicenjem da je tacno jedan od x0, x1, x2 jednak 1
x = [Int(f'x{i}') for i in range(3)]

# 1) Provera da li je x_hat legalno resenje
s = Solver()
for v in x:
    s.add(Or(v == 0, v == 1))
for i, bit in enumerate(x_hat):
    s.add(x[i] == bit)
s.add(Sum(x) == 1)

print(s.check())  # sat

# 2) Evaluacija ciljne funkcije na x_hat
hat_value = sum(weights[i] * x_hat[i] for i in range(3))
print(hat_value)  # 4

# 3) Pretraga za bolje dozvoljeno resenje
```

```
opt = Optimize()
for v in x:
    opt.add(Or(v == 0, v == 1))
opt.add(Sum(x) == 1)

objective = Sum([weights[i] * x[i] for i in range(3)])
h = opt.maximize(objective)

if opt.check() == sat:
    m = opt.model()
    best_x = [m.eval(x[i]).as_long() for i in range(3)]
    best_val = m.eval(objective).as_long()
    print(best_x)    # [1, 0, 0]
    print(best_val)  # 5
```

Листинг 8: Z3 провера кандидатног решења добијеног квантним или хеуристичким решавачем.

У овом примеру решење $\hat{\mathbf{x}}$ јесте легално, али није оптимално: Z3 налази бољи дозвољени битски низ и тиме пружа егзактан класични референтни резултат за поређење.

Напомена. У пракси је ово један од најважнијих хибридних образаца: QUBO се користи као формат за оптимизацију, а Z3 као формални слој који доказује да су казнене чланови коректни, да су коефицијенти довољни и да добијени битски низ заиста представља важеће решење полазног проблема.

7.2 Предлог верификационог тока рада

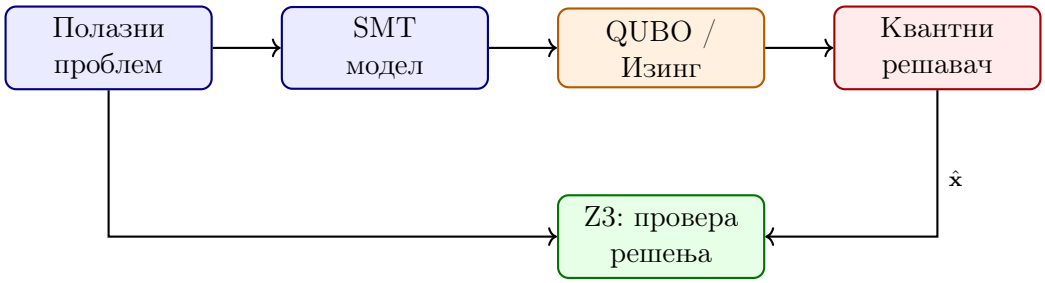
На основу претходне анализе, предлажемо следећи хибридни ток рада у коме SMT/SAT алати служе као верификациони слој на кључним тачкама QUBO тока рада:

1. **Формулација QUBO.** Проблем се кодира у QUBO матрицу Q и казнене чланове P_j.
2. **Верификација казнене семантике (Z3 / CVC5).** За сваки P_j аутоматски се доказује да је нула тачно на дозвољеним стањима и строго позитиван на кршењима (корак 1 из претходне секције). Алати попут Z3 или CVC5 могу се покренути аутоматски у оквиру процеса тестирања.
3. **Верификација скале казне (Z3 / Gurobi).** Проверава се да ли је изабрани λ довољан да ниједно недозвољено стање не подбаци у QUBO енергији испод оптималног дозвољеног решења. За мале инстанце то ради Z3; за веће се може користити MILP решавач (нпр. Gurobi или CPLEX) са

истим упитом.

4. **Квантни / хеуристички решавач.** Верификована QUBO матрица предаје се квантном анилеру, QAOA колу или симулираном жарењу.
5. **Верификација решења (Z3).** Враћени битски низ $\hat{\mathbf{x}}$ проверава се против оригиналних ограничења; Z3 истовремено тражи боље легално решење и тако мери квалитет апроксимације.

Овај образац не захтева потпуну формалну верификацију целог квантног алгоритма. Само три тачно дефинисана SMT упита довољна су да се елиминишу најчешћи извори грешке у QUBO пипелини.



Слика 3: Хибридни ток рада у коме се полазни ограничени проблем најпре преводи у SMT и QUBO/Изингову форму, затим решава квантно или хеуристички, а добијени кандидат се на крају проверава помоћу Z3.

8 Закључак

У овом раду посматрали смо MaxCut као репрезентативан НП-тежак проблем и показали да се исти задатак може описати на два комплементарна начина: симболички, преко ограничења и SMT модела, и оптимизационо, преко QUBO/Изингове формулације. Управо та двојност чини проблем погодним за повезивање аутоматског резоновања, класичних решавача и квантних алгоритама.

Најважнији резултат није само у томе што су приказани Z3 и QAOA као два различита приступа, већ у томе што је показано како их QUBO формализација повезује у јединствен ток рада. Z3 служи као прецизан алат за моделовање и егзактан решавач за мање инстанце, али и као алат за проверу да ли QUBO казнени чланови заиста изражавају полазна ограничења и да ли кандидатска решења добијена квантним или хеуристичким путем заиста задовољавају оригиналан проблем.

Кључни закључци:

- **MaxCut је добар мост између теорије и примене.** Он је довољно једноставан да се јасно формулише у SMT-у, али и довољно богат да природно води ка QUBO и Изинговом моделу.

- **QUBO је заједнички језик различитих метода.** Превођењем ограничења у квадратној бинарној форми добија се формулација погодна и за класичне хеуристике и за квантне оптимизационе алгоритме.
- **Z3 није само решавач, већ и верификатор.** У раду је показано да SMT решавач може да провери семантику казнених чланова, довољност коефицијената казне и исправност добијених битских ниски.
- **Квантни алгоритми су тренутно најпрактичнији у хибридном окружењу.** На данашњем хардверу QAOA, квантно жарење и сродне методе треба посматрати пре свега као део ширег процеса претраге, док симболичко резоновање обезбеђује поузданост и тумачење резултата.

Из те перспективе, аутоматско резоновање има двоструку улогу: оно је и језик за формално задавање проблема и механизам за независну проверу оптимизационих резултата.

Отворена питања

- **Превођење у QUBO:** Како SMT ограничења систематски преводити у QUBO уз формалну гаранцију коректности?
- **Избор казнених коефицијената:** Како аутоматски бирати довољно велике, али не и превелике коефицијенте казне, уз помоћ SMT верификације?
- **Хибридни токови рада:** Како најбоље комбиновати квантне или хеуристичке претраге са Z3 (и осталим претраживачима) тако да један систем генерише потенцијална решења, а други их филтрира и доказује?
- **Емпиријска поређења:** Потребна су шира поређења на већим скуповима инстанци како би се јасније видело када QUBO приступ и квантни решавачи доносе предност у односу на чисто симболичко решавање.

Иако су данашњи квантни уређаји и даље ограничени, веза између аутоматског резоновања, QUBO моделирања и квантне оптимизације већ сада представља плодан правац истраживања. Управо у тој комбинацији формалне прецизности и нових модела претраге лежи највећи потенцијал за будуће системе резоновања и оптимизације.

Литература

[1] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, LNCS 4963, pages 337–340. Springer, 2008. doi:10.1007/978-3-540-78800-3_24.

[2] N. S. Bjørner and A.-D. Phan. νZ — maximal satisfaction with Z3. In *Proceedings of the 6th International Symposium on Symbolic Computation in Software Science (SCSS 2014)*, pages 1–9, 2014.

[3] M. X. Goemans and D. P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995. doi:10.1145/227683.227684.

[4] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.

[6] F. Glover, G. Kochenberger, R. Hennig, and Y. Du. Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Annals of Operations Research*, 314:141–183, 2022. doi:10.1007/s10479-022-04634-2.

[7] V. V. Vazirani. *Approximation Algorithms*. Springer, 2001.

[8] A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors. *Handbook of Satisfiability*. IOS Press, 2009.

[9] H. Y. Wong. *Introduction to Quantum Computing: From a Layperson to a Programmer in 30 Steps*. Springer, 2022.

[10] C. C. McGeoch. *Adiabatic Quantum Computation and Quantum Annealing: Theory and Practice*. Synthesis Lectures on Quantum Computing. Morgan & Claypool Publishers, 2014. doi:10.2200/S00585ED1V01Y201407QMC008.

[11] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose. Quantum annealing with manufactured spins. *Nature*, 473:194–198, 2011. doi:10.1038/nature10012.

[12] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[13] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi:10.1126/science.220.4598.671.